A PROJECT REPORT ON

Retro Game Hub

BACHELOR OF TECHNOLOGY

IN

CSIT (CYBER SECURITY)

SUBMITTED BY

Group No: 3

PRN	Name of the Student
2501106112	Dristhi Galani
2501106113	Tanvee Thakre
2501106118	Atharva Tirkhunde
2501106136	Rishit Kinge
2501106155	Shubham Salunke

UNDER THE GUIDANCE OF

(Prof.Rahul Raut)



SCHOOL OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY SYMBIOSIS SKILLS AND PROFESSIONAL UNIVERSITY, PUNE ACADEMIC YEAR 2025-26



CERTIFICATE

This is to certify that the project titled "Retro Game Hub" submitted to the School of Computer Science and Information Technology (CSIT), Symbiosis Skills and Professional University, Pune, as part of the Project-Based Learning (PBL) is based on our original work carried out under the guidance of _____.

The report has not been submitted elsewhere for the award of any other degree, diploma, or certification.

The material borrowed from other sources and incorporated in the report has been duly acknowledged and referenced.

We understand that we could be held responsible and accountable for plagiarism, if any, detected later.

Group No:

Sr. No.	PRN	Name	Sign
1	2501106155	Shubham Salunke	_____
2	2501106112	Dristhi Galani	_____
3	2501106136	Rishit Kinge	_____
4	2501106113	Tanvee Thakre	_____
5	2501106118	Atharva Tirkhunde	_____

Date: _____

Place: Pune

Name of Guide
Designation

Director
School of CSIT

ABSTRACT

Retro Game Hub is a browser-based arcade platform that recreates classic games using only HTML, CSS, and Vanilla JavaScript. Built without frameworks or external libraries, it highlights strong fundamentals in front-end development while remaining lightweight, fast, and fully offline-capable. The entire application runs on the client side, ensuring accessibility across modern browsers like Chrome, Firefox, Safari, and Edge without requiring installation or internet connectivity after initial load.

The platform features nine fully playable games: Tic-Tac-Toe, Dots & Boxes, Snakes & Ladders, Battleship, Connect Four, Hangman, Memory Match, Tetris, and Pong. Each game is independently implemented with its own logic and mechanics but follows a consistent interface design, allowing smooth navigation and a unified user experience. A shared Heads-Up Display (HUD) provides real-time updates such as scores, player turns, and coin balance, ensuring clarity and engagement throughout gameplay.

A key component of Retro Game Hub is its persistent player profile system. Using browser storage (LocalStorage), the platform saves user progress, coin totals, and gameplay data, allowing users to return and continue seamlessly without needing accounts or servers. This is complemented by a centralized coin reward system, where players earn coins based on performance across all games. The reward pipeline has been optimized to ensure consistent and accurate coin distribution, including recent fixes for Tetris and Pong.

Artificial Intelligence is integrated into multiple games to provide engaging single-player experiences. The AI varies by game, ranging from simple rule-based logic to more adaptive strategies, ensuring balanced difficulty and responsiveness without heavy computation.

Recent updates have improved overall system reliability and maintainability. A unified game-over modal system standardizes end-of-game interactions, while the removal of unused modules has streamlined the codebase. Enhancements to data persistence ensure more reliable storage and retrieval of player information.

With its retro-inspired design, responsive layout, and zero-dependency architecture, Retro Game Hub demonstrates how classic games can be transformed into a modern, cohesive web application using only core web technologies.

Keywords: *Retro games, browser arcade, HTML CSS JavaScript, Vanilla JS, offline web app, LocalStorage, coin system, AI games, responsive design, classic games, frontend project*

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who supported, guided, and encouraged us throughout the development of the Retro Game Hub project.

First and foremost, we are deeply thankful to our project guide for their invaluable guidance, consistent encouragement, and constructive feedback at every stage of this project. Their insights into front-end development, application design, and user experience played a crucial role in shaping the structure and quality of our work.

We extend our heartfelt thanks to the Director and faculty members of the School of Computer Science and Information Technology at Symbiosis Skills and Professional University, Pune, for fostering an environment that promotes innovation, creativity, and practical learning. The foundational knowledge gained during our coursework — particularly in web technologies such as HTML, CSS, and JavaScript — enabled us to conceptualize and successfully build this browser-based gaming platform.

We would also like to acknowledge the broader web development and open-source community for providing valuable learning resources, documentation, and references that guided us in implementing game logic, browser storage mechanisms like LocalStorage, and responsive design principles. These resources were instrumental in helping us develop features such as AI-based gameplay, persistent player profiles, and the coin reward system without relying on external libraries or frameworks.

Finally, we express our sincere appreciation to our families and peers for their continuous moral support, encouragement, and patience throughout this project. Their motivation helped us stay focused and overcome challenges during the development process.

CHAPTER 1: INTRODUCTION

1.1 Background

Retro Game Hub is a web-based game collection inspired by the notebook games played by students during their school years — Tic-Tac-Toe, Dots & Boxes, Snakes & Ladders, and more. The project aims to recreate these classics in a modern browser environment while preserving their simplicity and nostalgic appeal. The application runs directly in any modern browser without installation, a server, or any external library dependency.

The entire codebase spans 13 files and approximately 4,200 lines of code, covering a modular JavaScript architecture, responsive CSS theming, a procedural sound engine, and individual game modules each encapsulating their own logic, AI, and reward handling.

1.2 Motivation

The motivation behind the project is to build a compact, offline-capable gaming platform that works without installation and showcases core web technologies. It also serves as a practical demonstration of game logic, UI design, state persistence using browser LocalStorage, and artificial intelligence through algorithms such as Minimax and Alpha-Beta pruning.

The team was inspired by the universality of simple pencil-and-paper games and wanted to honour that tradition in a digital form that is accessible to anyone with a browser, on any device, anywhere — even without internet access after the initial load.

1.3 Problem Statement

Many notebook-style games are available only as isolated mobile applications or desktop installs, often bloated with advertisements and requiring an account or a data connection. There is a gap for a lightweight, browser-accessible retro game hub that can run on any device with minimum dependencies, provides a unified player profile system, and works offline. This project addresses that gap.

1.4 Objectives

- Build a multi-game web application using only HTML, CSS, and Vanilla JavaScript.
- Implement persistent player profiles and reward tracking using LocalStorage.
- Create AI opponents and game logic for multiple classic games.
- Provide a polished retro interface with sound, themes, and responsive design.
- Fix known gameplay issues including reward persistence and modal behaviour.
- Ensure the application is fully offline-capable after the first load.

1.5 Scope

The scope of this project includes nine playable games (Tic-Tac-Toe, Dots & Boxes, Snakes & Ladders, Battleship, Connect Four, Hangman, Memory Match, Tetris, and Pong), player account management with PIN authentication, a coin reward economy, per-game and global leaderboards, four retro display themes, a procedural sound engine with seven music tracks and twelve sound effects, and a retro-themed responsive UI suitable for desktop and mobile browsers ranging from 360 px to 1440 px wide.

CHAPTER 2: LITERATURE REVIEW

2.1 Existing Solutions

Existing retro game hubs on the web typically fall into one of three categories. First, single-game implementations that focus on one classic title (e.g., many standalone Tetris or Tic-Tac-Toe clones) without any unified player system or progression. Second, framework-dependent multi-game portals that rely on React, Angular, or similar libraries, making them heavier and harder to host as a single static folder. Third, server-dependent platforms that require a back end to store player data, meaning they do not work offline and need an internet connection to function.

None of these categories satisfactorily combines lightweight offline capability, multiple classic games, persistent player data, a reward economy, and AI opponents in a single vanilla-JavaScript application.

2.2 Comparative Study

Retro Game Hub differentiates itself by using only vanilla JavaScript and local browser storage, combining multiple classic games in a single app while keeping the experience lightweight and offline-capable. Table 1 below summarises key differentiators:

Feature	Standalone Clones	Framework Portals	Server Platforms	Retro Game Hub
No install required	Yes	Yes	Yes	Yes
Multiple games	No	Yes	Yes	Yes
Works offline	Usually	No	No	Yes
Player profiles	No	Varies	Yes	Yes
AI opponents	Sometimes	Varies	Varies	Yes
Zero dependencies	Yes	No	No	Yes

Table 1: Comparative Study of Retro Game Hub vs Existing Solutions

CHAPTER 3: SUBJECT-WISE INTEGRATION

The Retro Game Hub project integrates concepts from multiple disciplines studied in the BTech CSIT programme. Table 2 below maps each course to the concepts applied and their specific use within the project.

Course Code	Course Name	Concept Used	Application in Project
CS23201	Data Structures & Algorithms	Arrays, Stacks, Queues, Game Trees	Used for game state management, board representation (Tic-Tac-Toe, Connect Four), and implementation of AI algorithms like Minimax using tree structures.
CS23202	Web Technologies & Vulnerabilities	HTML, CSS, JavaScript, LocalStorage, Web Security Basics	Used to build the complete frontend UI, responsive design, and secure handling of player data (PIN-based login, avoiding plain text storage).
CS23203	OS, Compiler & Security Engineering	Process Flow, Memory Management, Security Principles	Helps in structuring modular code, efficient execution of game loops, and implementing secure data handling mechanisms like hashed PIN storage.
CS23204	Human Behavior	User Experience (UX), Cognitive Design, Interaction Principles	Applied in designing intuitive UI, retro themes, sound feedback, and ensuring engaging gameplay experience for users.
CS23205	Python Programming	Logic Building, Algorithm Design, Problem Solving	Core programming concepts (though implemented in JavaScript) were derived from Python-based logic building such as loops, conditionals, and modular coding practices.
CS23206	Applied Mathematics - II	Probability, Logic, Graph Theory, Optimization	Used in AI decision-making (Minimax algorithm), probability-based game mechanics (dice in Snakes & Ladders), and scoring/reward calculations.

Table 2: Subject-Wise Integration of Course Concepts into Project

CHAPTER 4: METHODOLOGY & SYSTEM DESIGN

4.1 Architecture Diagram

The application follows a modular single-page application (SPA) architecture. A central app controller (main.js) orchestrates boot sequences, screen routing, and game launches. All screens — boot, login, home, game, leaderboard, settings — are defined as divs inside a single index.html and toggled by the UI module. Individual game modules expose a standard init/restart interface.

The core component hierarchy is as follows:

- index.html — Main app shell containing all 6 screens
- css/main.css — All styles, themes, animations, and CRT effects
- js/main.js — App controller, boot sequence, and routing logic
- js/player.js — LocalStorage-based player database and coin management
- js/ui.js — Screen manager, modals, HUD, and music indicator
- js/sound.js — Web Audio procedural music and sound effects engine
- games/*.js — Nine individual game modules (tictactoe, dots, snl, battleship, connectfour, hangman, memorymatch, tetris, pong)

4.2 Hardware Requirement

No special hardware is required. The application runs in any modern browser on a desktop computer, laptop, tablet, or smartphone. A minimum screen width of 360 px is supported. Audio output requires a sound card, but the application functions silently without one.

4.3 Software Requirement

Requirement	Details
Operating System	Any OS with a modern browser (Windows, macOS, Linux, Android, iOS)
Browser	Chrome 100+, Firefox 100+, Edge 100+, Safari 15+, Samsung Internet 20+
JavaScript	Must be enabled (ES6+)
Server / Database	None required — uses browser LocalStorage
Installation	None — open index.html directly
Internet	Not required after first load

Table 3: Software Requirements

4.4 Flowchart / Algorithm

Game flow begins at the animated boot screen, proceeds to the Player Station (login, register, or guest), then loads the Home screen displaying all game cards. On selecting a game the user chooses VS AI or VS Human, after which the relevant game module initialises. Each game module handles its own state machine, win/loss detection, and reward logic, then calls UI.showWinModal() on game end.

Key algorithms employed:

- Minimax (full-tree) for Tic-Tac-Toe — evaluates all 362,880 reachable states to play perfectly.
- Alpha-Beta Minimax at depth 5 for Connect Four — prunes unreachable branches for efficiency.
- Heuristic 3-tier strategy for Dots & Boxes — capture > safe move > random.
- Hunt & Target algorithm for Battleship AI — random search transitioning to systematic destruction on a hit.

4.5 Database Design

The project uses the browser's LocalStorage API as a lightweight key-value store. No external database is required. Each player record is stored as a JSON-serialised object under the key retro_players and indexed by player ID. The schema for each player object is:

- id — Unique player ID (3–16 alphanumeric characters)
- pinHash — SHA-like hash of the 4-digit PIN (never stored in plain text)
- avatar — Chosen emoji avatar from a set of 24
- coins — Running total of earned coins
- wins — Object mapping game key to win count plus total
- games — Object mapping game key to play count plus total
- streak — Current consecutive win count for streak bonus calculation

4.6 Models / Equations

Scoring and reward calculations are implemented using game-specific fixed coin values plus universal bonuses:

- Win reward: fixed value per game (10–25 coins depending on difficulty)
- Participation reward: +2 coins for any completed game (win or lose)
- Streak bonus: +5 coins when a player achieves 3 consecutive wins

4.7 Approach, Architecture, Tools & Technologies

The project was developed iteratively using a feature-branch workflow. Each game module was built and tested in isolation before being integrated into the main application. Bug-fix sprints addressed coin persistence, modal unification, and keyboard listener deduplication after the initial integration phase.

Technologies used: HTML5, CSS3 (custom properties / variables), ES6+ Vanilla JavaScript, Web Audio API (procedural sound), LocalStorage API, SVG (Dots & Boxes board). Development tools: VS Code, Chrome DevTools, Git.

CHAPTER 5: IMPLEMENTATION

5.1 Development Process

The application was built iteratively over multiple sprints. Sprint 1 established the SPA shell, player system, and UI manager. Sprint 2 added the first three games (Tic-Tac-Toe, Dots & Boxes, Snakes & Ladders) and the coin system. Sprint 3 added Battleship, Connect Four, Hangman, and Memory Match. Sprint 4 added Tetris and Pong. A final bug-fix sprint addressed coin persistence issues in Tetris and Pong, unified the game-over modal across all games, removed the obsolete Mastermind module, and corrected duplicate keyboard listener registration.

5.2 Modules

The application is composed of the following modules:

- UI Controller (ui.js) — screen navigation, modals, HUD updates, music track indicator
- Player Database (player.js) — account creation and login with PIN hashing, coin awarding, leaderboard data, game statistics recording
- Sound Engine (sound.js) — Web Audio API oscillator-based sound effects and 7 background music tracks with melody, bass, and harmony layers
- App Controller (main.js) — boot animation, screen routing, game launch dispatcher, mode selection
- Tic-Tac-Toe (tictactoe.js) — full Minimax AI, 3x3 board, win animation
- Dots & Boxes (dots.js) — SVG board, heuristic AI, extra-turn mechanic
- Snakes & Ladders (snl.js) — 100-square board, animated dice, 2-4 player support
- Battleship (battleship.js) — ship placement phase, Hunt/Target AI
- Connect Four (connectfour.js) — animated disc drop, Alpha-Beta AI at depth 5
- Hangman (hangman.js) — 10 categories, progressive gallow drawing, hint system
- Memory Match (memorymatch.js) — 4x4 emoji card grid, pair-matching logic
- Tetris (tetris.js) — 7 tetrominoes, line clearing, speed progression, next-piece preview
- Pong (pong.js) — canvas physics, mouse/touch paddle, first-to-7 scoring, AI tracking

5.3 Algorithms

Minimax for Tic-Tac-Toe: The algorithm recursively explores every possible game state from the current board position. It assigns +10 to an AI win, -10 to an AI loss, and 0 to a draw, then propagates values up the tree using max (AI turn) and min (player turn) selection. Because Tic-Tac-Toe has at most 9 moves the full tree is always evaluated, making the AI unbeatable.

Alpha-Beta pruning for Connect Four: The same Minimax principle is applied to a 6×7 board with depth limited to 5. Alpha-Beta pruning eliminates branches that cannot affect the final decision, reducing the number of nodes evaluated from O(b^d) to approximately O(b^(d/2)) in the best case. A heuristic scoring function rewards centre column control and open sequences of 2, 3, or 4 discs.

Hunt & Target for Battleship: The AI begins in Hunt mode, firing at random non-adjacent cells. On scoring a hit it switches to Target mode, firing at the four orthogonal neighbours of the hit cell in sequence until the ship is sunk, then reverts to Hunt mode.

Heuristic AI for Dots & Boxes: The AI evaluates all available lines in three passes — first capturing any box that can be completed, then taking any safe line that does not create an open 3-sided box for the opponent, and finally choosing at random if no better option exists.

CHAPTER 6: RESULT ANALYSIS

6.1 Output

The final product is a fully functional browser-based retro arcade. Key output characteristics:

- Nine playable games accessible from a single home screen
- Player profiles with PIN login, coin balance, and per-game win statistics
- Global and per-game leaderboards showing top 20 players
- Coin rewards correctly credited on win for all nine games
- Game-over modal displayed consistently after every game with result, coins awarded, and Play Again / Home options
- Four visual themes switchable without page reload
- Procedurally synthesised background music and sound effects with no audio files
- Responsive layout from 360 px mobile to 1440 px desktop

6.2 Testing

Test Case	Expected Result	Outcome
Player login and guest mode	Both modes accessible	Pass
Coin award on Tetris win	+20 coins added	Pass (after fix)
Coin award on Pong win	+12 coins added	Pass (after fix)
Game-over modal for all games	Modal appears with result	Pass
Duplicate keyboard listener (Tetris)	No double-move on restart	Pass (after fix)
LocalStorage persistence	Coins survive page reload	Pass
Theme switching	Instant, no reload	Pass
Responsive layout (360px)	No overflow	Pass
Offline use after first load	All features work	Pass

Table 4: Test Cases and Results

6.3 Performance

The application is extremely lightweight. Page load time is under 200 ms on a standard connection. All 13 files total less than 100 KB. There are zero external HTTP requests at runtime. The Web Audio API synthesises all sound in real time, contributing zero bytes of audio asset payload. Memory usage in Chrome during gameplay was observed to remain below 30 MB.

Metric	Value
Total files	13
Lines of code	~4,200
External dependencies	0
Audio files	0
Page load time	< 200 ms
Min screen size supported	360 px
Supported browsers	5 (Chrome, Firefox, Edge, Safari, Samsung Internet)
Background music tracks	7
Sound effects	12
Display themes	4

Table 5: Performance Metrics

CHAPTER 7: DISCUSSION

This project demonstrates that classic games can be effectively reimagined within a modern web browser using only core technologies — HTML, CSS, and Vanilla JavaScript — without compromising on depth, interactivity, or user experience. By deliberately avoiding frameworks and external libraries, the development process emphasized a strong understanding of JavaScript fundamentals, including the event-driven model, efficient DOM manipulation, and advanced browser capabilities such as the Web Audio API. This approach not only kept the application lightweight and fast but also ensured complete control over architecture and performance.

One of the most critical technical challenges encountered during development was maintaining a consistent and reliable coin reward system across all nine games. In earlier iterations, reward logic was fragmented, with each game handling outcomes independently, leading to inconsistencies and bugs in coin allocation. This issue was resolved by centralizing the reward mechanism through a unified function, PlayerDB.awardCoins(), followed by synchronized UI updates using UI.updateHUD(). Additionally, all game-ending scenarios were standardized through a single modal system using UI.showWinModal(). This architectural refinement significantly improved both reliability and maintainability, transforming the codebase into a more modular and scalable system.

Another standout feature of the project is the procedural sound engine. Instead of relying on preloaded audio files, all sound effects and background music were generated dynamically using oscillators from the Web Audio API. This design choice eliminated the need for audio assets, reducing load times and overall application size to near zero in terms of media weight. Despite this, the platform delivers a rich audio experience, with multiple layered tracks — melody, bass, and harmony — creating distinct sound identities for different games and enhancing immersion.

The inclusion of a guest mode reflects a thoughtful user-centric design strategy. It allows users to explore and play all available games without requiring immediate registration, thereby lowering the barrier to entry. At the same time, features such as the coin counter and leaderboard remain visible but restricted, encouraging users to create a profile to unlock full functionality. This approach aligns with widely adopted free-to-play design patterns, where accessibility is balanced with incentive-driven engagement.

Overall, the project highlights how thoughtful architecture, efficient state management, and creative use of native browser APIs can transform simple, traditional games into a cohesive and polished digital arcade experience.

CHAPTER 8: CONCLUSION

Retro Game Hub successfully delivers a complete browser-based arcade experience by integrating multiple classic games into a single, cohesive platform. The application combines essential gaming features such as persistent player profiles, a coin-based reward economy, AI-driven opponents, procedural audio generation, and a retro-inspired user interface — all developed using only HTML, CSS, and Vanilla JavaScript. Notably, the entire system operates without any external dependencies or server-side infrastructure, demonstrating the capability of core web technologies to support complex, interactive applications.

The project has effectively met all its initial objectives. All nine games are fully functional and playable within the browser, each maintaining consistent design and interaction patterns. The player profile system ensures that user data, including coin balances and progress, persists reliably across sessions using browser storage. AI opponents across various games provide a balanced and engaging challenge, enhancing single-player gameplay. The retro-themed interface is both visually appealing and responsive, adapting seamlessly across different screen sizes and devices. Additionally, prior issues related to reward distribution and inconsistent game-over handling have been resolved through architectural improvements, resulting in a more stable and maintainable codebase.

FUTURE SCOPE

Looking ahead, the project offers significant scope for further enhancement and scalability. Phase 2 will shift focus towards introducing real-time multiplayer functionality using WebSocket-based communication, allowing players to connect via simple room codes. Additional games such as Rock-Paper-Scissors and Sudoku are planned to expand the platform's variety. The implementation of a Progressive Web App (PWA) framework will enable users to install the platform on mobile devices, improving accessibility and engagement.

Phase 3 will concentrate on enriching the overall user experience through advanced gamification features. Planned additions include an achievement badge system, daily challenge modes that rotate games every 24 hours, and customizable in-game elements such as unlockable skins purchasable using coins. Competitive features like global tournament brackets and expanded multiplayer modes — including support for up to four players — are also envisioned. Furthermore, cloud-based data storage using services such as Firebase or Supabase will allow cross-device synchronization of player profiles, enhancing continuity and convenience.

In terms of deployment, the groundwork for mobile integration has already been established through the creation of a PWA manifest file with appropriate icon assets. Future development will involve completing service worker implementation and packaging the application for distribution on platforms like the Google Play Store using Trusted Web Activity (TWA).

Overall, Retro Game Hub not only achieves its current goals but also lays a strong foundation for future expansion, positioning it as a scalable and feature-rich browser gaming platform.

REFERENCES

[1] MDN Web Docs — HTML, CSS, and JavaScript Reference. Mozilla Developer Network. https://developer.mozilla.org

[2] Web Audio API Specification. W3C Working Draft. https://www.w3.org/TR/webaudio/

[3] Using the Web Storage API (LocalStorage). MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

[4] Russell, S. and Norvig, P. (2020). Artificial Intelligence: A Modern Approach, 4th ed. Pearson.

[5] Minimax Algorithm in Game Theory. GeeksforGeeks. https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-1-introduction/

[6] Alpha-Beta Pruning. GeeksforGeeks. https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-4-alpha-beta-pruning/

[7] SVG — Scalable Vector Graphics. W3C. https://www.w3.org/Graphics/SVG/

[8] Progressive Web Apps (PWA). Google Developers. https://web.dev/progressive-web-apps/

APPENDIX

A. Folder Structure

```
retro-game-hub/
  index.html — Main app shell (all 6 screens)
  css/
    main.css — All styles, themes, animations, CRT effects
  js/
    sound.js — Web Audio sound engine + 7 music tracks
    player.js — Player database (LocalStorage)
    ui.js — Screen manager, modals, HUD, music indicator
    main.js — App controller, boot sequence, routing
  games/
    tictactoe.js — Tic-Tac-Toe + Minimax AI
    dots.js — Dots & Boxes + heuristic AI
    snl.js — Snakes & Ladders + dice engine
    battleship.js — Battleship + Hunt/Target AI
    connectfour.js — Connect Four + Alpha-Beta Minimax AI
    hangman.js — Hangman + hint system
    memorymatch.js — Memory Match + card flipping
    tetris.js — Tetris + line clearing
    pong.js — Pong + paddle battle
  README.md
```

B. Game Coin Reward Summary

Action	Coins Earned
Win Tic-Tac-Toe	+10
Win Connect Four	+12
Win Pong	+12
Win Hangman	+14
Win Dots & Boxes	+15
Win Memory Match	+16
Win Tetris	+20
Win Snakes & Ladders	+20
Win Battleship	+25
Play any game (win or lose)	+2
3-win streak bonus	+5

Table 6: Game Coin Reward Summary

— End of Report —

Built with HTML, CSS, and Vanilla JavaScript | SSPU Pune | BTech CSIT 2025-26